
Guida autonoma in TORCS

mediante behavioral cloning

Diagnosi, controllo manuale e raccolta dati per l'addestramento di un agente di guida

Circuito: Corkscrew | Driver: scr_server

Gruppo: _____

Componenti: _____

Anno accademico: _____

Indice

1	Introduzione e obiettivi	2
	1.1 Articolazione del progetto	2
2	Architettura dell'ambiente	2
	2.1 Sensori e comandi	2
3	Diagnosi del problema di avvio	3
	3.1 Causa individuata	3
	3.2 Verifiche operative	3
4	Controllo manuale: progettazione e correzioni	4
	4.1 Cambio marcia automatico	4
	4.2 Accelerazione più reattiva	4
	4.3 Sterzo corretto e stabilizzazione	4
	4.4 Registrazione dei dati	4
5	Analisi del giro registrato	5
	5.1 Qualità dei dati e pulizia necessaria	6
6	Metodologia di apprendimento (lavoro futuro)	6
	6.1 Behavioral cloning	6
	6.2 Modello di base previsto: K-Nearest Neighbors	6
	6.3 Estensione prevista: rete neurale	6
	6.4 Criticità nota: distribution mismatch	6
	6.5 Architettura prevista per la guida autonoma	7
7	Consegna e gestione su GitHub	7
8	Stato di avanzamento e prossimi passi	7
9	Conclusioni	7

1 Introduzione e obiettivi

Il presente documento descrive il lavoro svolto nell'ambito della competizione IBM AI Racing League, organizzata dall'Università degli Studi di Salerno in collaborazione con IBM SkillsBuild e Codemotion. L'obiettivo della competizione è lo sviluppo di un agente di intelligenza artificiale capace di guidare in modo autonomo un'automobile all'interno del simulatore TORCS (The Open Racing Car Simulator), nella sua versione controllabile via Python, percorrendo il circuito Corkscrew nel minor tempo possibile, senza uscire di pista e senza perdere aderenza.

Il veicolo è pilotato attraverso il driver `scr_server`, che espone i sensori del veicolo e accetta i comandi di controllo tramite una connessione di rete (socket UDP). La strategia adottata dal gruppo è il behavioral cloning (clonazione del comportamento), un approccio di apprendimento supervisionato in cui un modello impara a guidare osservando i dati di guida prodotti da un pilota umano.

1.1 Articolazione del progetto

Il progetto si articola in tre fasi logiche:

1. Raccolta dati. Un operatore umano guida il veicolo manualmente registrando, a ogni istante, lo stato dei sensori e l'azione di controllo corrispondente.
2. Addestramento. Un modello di apprendimento viene addestrato sui dati raccolti, affinché impari ad associare ogni stato sensoriale alla relativa azione.
3. Guida autonoma. Il modello addestrato sostituisce il pilota umano e controlla il veicolo in autonomia.

Alla data di redazione, la Fase 1 è operativa e ha prodotto i primi dati di guida; le Fasi 2 e 3 sono progettate e descritte nella loro metodologia, ma non ancora implementate. Tali sezioni sono chiaramente contrassegnate come lavoro futuro.

2 Architettura dell'ambiente

L'ambiente di lavoro è costituito da TORCS in esecuzione su sistema Windows e da un insieme di moduli Python che comunicano con il simulatore. I componenti rilevanti per il progetto sono i seguenti.

- `snakeoil3_jm2.py` — client di rete che apre una socket UDP (porta 3001), riceve dal simulatore lo stato del veicolo (stringa di sensori) e invia i comandi di controllo. Contiene una correzione specifica per Windows che ripulisce i caratteri di controllo (`\x00`) presenti nelle stringhe ricevute.
- `manual_control.py` — modulo di guida manuale che legge la tastiera, traduce i tasti in comandi di sterzo, accelerazione, freno e marcia, e registra ogni passo in un file di log.
- `main.py` — una seconda versione del controllo manuale, basata sulla libreria `msvcrt`; come discusso nella Sezione [3](#), questa versione presenta una limitazione che ne ha impedito l'uso.

2.1 Sensori e comandi

Lo stato osservabile del veicolo, utilizzato come input del modello, comprende le grandezze riportate nella Tabella [1](#). I comandi di controllo (output) sono lo sterzo, l'accelerazione, il freno e la marcia.

Tabella 1: Principali grandezze sensoriali e comandi del veicolo.

Grandezza	Tipo	Significato
speedX	sensore	velocità longitudinale (km/h)
angle	sensore	angolo del veicolo rispetto all'asse pista
trackPos	sensore	posizione laterale rispetto al centro pista
track[0..18]	sensore	19 distanze dal bordo pista (telemetri)
rpm	sensore	regime motore
steer	comando sterzo, in $[-1, 1]$ (positivo = sinistra)	
accel	comando accelerazione, in $[0, 1]$	
brake	comando freno, in $[0, 1]$	
gear	comando marcia	

Si segnala che nel repository è presente anche l'ambiente `gym_torcs.py` (con `example_experiment.py` e `sample_agent.py`): si tratta di un wrapper pensato per il reinforcement learning su Linux, che utilizza una porta differente (3101) e un agente ad azioni casuali. Tale ambiente non è stato impiegato, poiché l'approccio scelto è il behavioral cloning sulla porta 3001.

3 Diagnosi del problema di avvio

La prima difficoltà tecnica affrontata è stata l'impossibilità, per uno dei membri del gruppo, di far muovere il veicolo: all'avvio della simulazione l'automobile rimaneva ferma al punto di partenza, mentre un altro componente del gruppo riusciva a guidare regolarmente.

3.1 Causa individuata

L'analisi ha rivelato che i due membri utilizzavano due moduli differenti per la lettura della tastiera:

- `main.py` impiega la libreria `msvcrt`, che intercetta la tastiera solo se la finestra attiva è il prompt dei comandi. Quando l'utente porta in primo piano la finestra di TORCS per guidare, il prompt perde il fuoco, i tasti non vengono più letti e il comando di accelerazione resta a zero: il veicolo non parte.
- `manual_control.py` impiega la libreria `pynput`, che cattura la tastiera a livello di sistema operativo, e quindi funziona correttamente anche con TORCS in primo piano.

La causa del malfunzionamento non era dunque un errore casuale, ma una differenza strutturale tra i due metodi di acquisizione dell'input. La soluzione adottata è stata l'uso esclusivo di `manual_control.py` basato su `pynput`.

3.2 Verifiche operative

Sono stati inoltre verificati i prerequisiti di avvio corretto, che si sono rivelati condizioni necessarie al funzionamento:

1. TORCS deve essere avviato prima dello script Python e deve trovarsi in attesa di connessione (schermata di attesa del server);
2. il driver `scr_server` deve essere configurato sulla stessa porta usata dallo script (3001);
3. la libreria `pynput` deve essere installata (`pip install pynput`).

4 Controllo manuale: progettazione e correzioni

A partire dalla versione funzionante di `manual_control.py`, sono state introdotte alcune correzioni mirate, con due obiettivi: rendere la guida più stabile e veloce, e garantire che i dati registrati fossero adatti al successivo addestramento. Le modifiche hanno riguardato esclusivamente il file `manual_control.py`; nessun altro modulo è stato alterato.

4.1 Cambio marcia automatico

Nella versione originale il cambio era manuale (tasti W/S), con il rischio di portare per errore il veicolo in retromarcia o di bloccarlo. È stato introdotto un cambio automatico in funzione della velocità, che elimina l'errore umano e semplifica la guida:

```
1 def auto_gear(self, speed, rpm):
2     if speed < 40:         return 1
3     elif speed < 80:      return 2
4     elif speed < 115:    return 3
5     elif speed < 150:    return 4
6     elif speed < 185:    return 5
7     else:                 return 6
```

4.2 Accelerazione più reattiva

Il coefficiente di smorzamento (`smooth`) dell'accelerazione è stato aumentato da 0,1 a 0,25, in modo che il veicolo raggiunga rapidamente la piena accelerazione invece di salire lentamente. È stato inoltre impedito l'invio simultaneo di accelerazione e freno, causa tipica di stallo del veicolo:

```
1 target_accel = 1.0 if Key.up in self.keys else 0.0
2 self.state['accel'] += (target_accel - self.state['accel']) * 0.25
3 # Se si accelera, il freno viene azzerato:
4 if target_accel > 0.0:
5     self.state['brake'] = 0.0
```

4.3 Sterzo corretto e stabilizzazione

In TORCS lo sterzo positivo corrisponde a una svolta a sinistra. La logica di stabilizzazione è stata rivista: quando il pilota non sterza, un piccolo termine correttivo basato su `angle` e `trackPos` riporta automaticamente il veicolo verso il centro pista, riducendo il rischio di uscita. Il coefficiente di risposta è stato aumentato (da 0,2 a 0,35) per affrontare i tornanti stretti del Corkscrew:

```
1 max_steer = max(0.35, 1.0 - speed / 250.0)
2 steer_input *= max_steer
3 if abs(steer_input) < 0.01:
4     # Nessun input: riporta l'auto verso il centro pista
5     steer_target = (angle * 0.5) - (trackPos * 0.1)
6 else:
7     steer_target = steer_input
8 self.state['steer'] += (steer_target - self.state['steer']) * 0.35
```

4.4 Registrazione dei dati

A ogni passo della simulazione (intervallo di 0,02 s) lo script registra lo stato dei sensori e l'azione corrispondente, in due formati: un file CSV (`manual_log.csv`) e un file JSON (`manual_log.json`).

Il formato dei log è stato lasciato invariato rispetto alla versione originale, in quanto costituisce il dataset per la fase di addestramento. Ogni riga del CSV rappresenta un esempio di addestramento nella forma stato → azione.

5 Analisi del giro registrato

Il primo giro registrato manualmente ha prodotto un dataset utilizzabile come base per l'addestramento. La Figura [1](#) mostra il profilo di velocità lungo il giro, ricostruito dai dati del log.

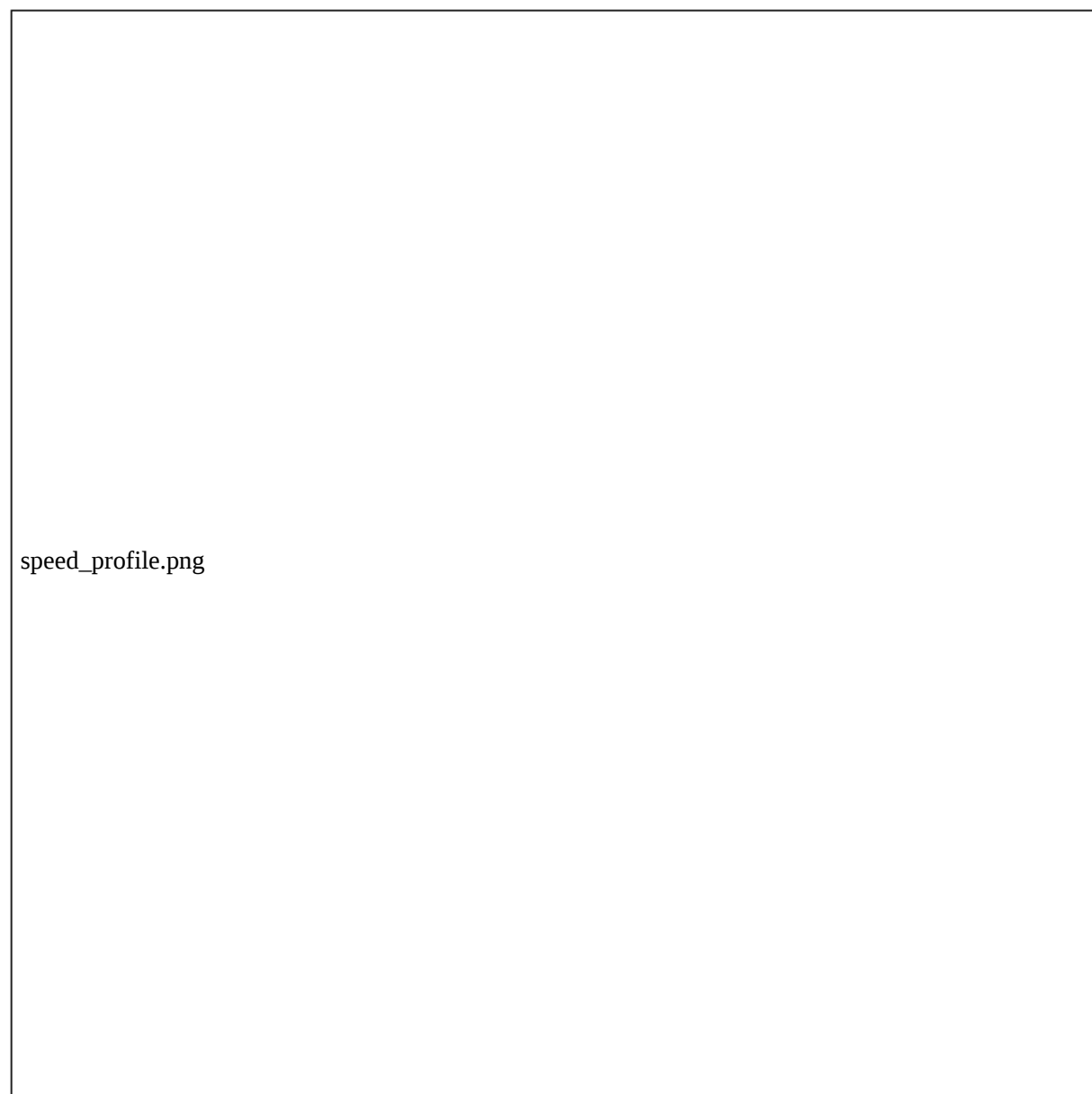


Figura 1: Profilo di velocità (speedX) durante il giro registrato manualmente. Le fasi di accelerazione e di frenata in ingresso curva sono chiaramente distinguibili; il picco di velocità raggiunto è di circa 286 km/h.

5.1 Qualità dei dati e pulizia necessaria

L'analisi del log ha evidenziato un aspetto rilevante per la fase di addestramento: le righe finali del file contengono valori degeneri (comandi prossimi allo zero macchina, dell'ordine di 10–323, con stato del veicolo “congelato”), corrispondenti al momento in cui il giro era di fatto concluso o il veicolo bloccato. Inoltre, in alcuni istanti i sensori track assumono il valore -1 , che indica veicolo fuori pista.

Nota metodologica (pre-addestramento). Prima di addestrare il modello sarà necessario filtrare queste righe: gli esempi degeneri o di fuori pista non rappresentano un comportamento di guida valido e, se inclusi, insegnerebbero al modello a non reagire. Il filtraggio consiste nello scartare le righe con tutti i sensori track pari a -1 e quelle con comandi numericamente nulli.

6 Metodologia di apprendimento (lavoro futuro)

Le sezioni seguenti descrivono la metodologia progettata per le Fasi 2 e 3. Tali fasi non sono ancora state implementate alla data di redazione del presente documento.

6.1 Behavioral cloning

Il behavioral cloning riformula la guida come un problema di apprendimento supervisionato: a partire dal dataset di coppie (stato, azione) raccolto nella Fase 1, si addestra un modello f tale che, dato uno stato sensoriale s , esso predica l'azione $a = f(s)$ che il pilota umano avrebbe compiuto. Il modello non ottimizza un obiettivo di gara, ma imita il comportamento osservato.

6.2 Modello di base previsto: K-Nearest Neighbors

In accordo con il materiale didattico del corso, il modello di partenza previsto è il K-Nearest Neighbors (K-NN). Si tratta di un metodo basato sulla memoria: non richiede un addestramento in senso stretto, ma conserva gli esempi raccolti e, in fase di guida, per ogni stato corrente individua gli esempi più simili già osservati e ne riproduce l'azione associata. I vantaggi attesi sono la semplicità implementativa e la trasparenza del comportamento; il limite principale è il costo computazionale che cresce con il numero di esempi memorizzati.

6.3 Estensione prevista: rete neurale

Come possibile evoluzione è prevista una rete neurale: a differenza del K-NN, essa “comprime” gli esempi in una funzione parametrica appresa, con una migliore capacità di generalizzazione a stati non osservati e un costo di predizione costante. Tale estensione sarà valutata qualora le prestazioni del K-NN non risultassero sufficienti.

6.4 Criticità nota: distribution mismatch

Una criticità tipica del behavioral cloning, da tenere in considerazione, è il disallineamento di distribuzione: il modello si comporta correttamente solo negli stati simili a quelli presenti nel dataset. Se durante la gara il veicolo entra in stati molto diversi da quelli registrati, le predizioni possono essere errate. La strategia di mitigazione prevista consiste nel raccogliere giri vari e puliti e nel registrare manovre di recupero (avvicinamento al bordo e successivo raddrizzamento), così da arricchire il dataset con situazioni di correzione.

6.5 Architettura prevista per la guida autonoma

La Fase 3 prevede un nuovo script (ad es. `auto_drive.py`) con la medesima struttura di `manual_control.py` — stessa connessione tramite `snakeoil3_jm2.py` e medesimo ciclo di controllo — in cui la lettura della tastiera è sostituita dalle predizioni del modello addestrato. Si sottolinea che il modulo di connessione resterà invariato: la sostituzione riguarda unicamente la sorgente dei comandi.

7 Consegna e gestione su GitHub

I requisiti della competizione prevedono la consegna del progetto TORCS in Python, di due video (il giro più veloce con partenza da fermo e una presentazione del gruppo in lingua inglese della durata massima di tre minuti), di un repository GitHub e della raccolta dei badge IBM SkillsBuild.

Il materiale condiviso dovrà essere accessibile senza password e non modificabile dopo la consegna finale. Per soddisfare tali requisiti si prevede di pubblicare il repository in modalità pubblica e di “congelare” lo stato finale tramite un tag di versione (o una release), che fissa in modo verificabile il contenuto alla data di consegna senza impedirne la consultazione.

Da completare: pubblicazione del repository pubblico, inserimento del codice (`manual_control.py`, `snakeoil3_jm2.py`, gli script di addestramento e di guida autonoma una volta realizzati) e dei dati di log, creazione del tag/release di consegna, caricamento dei due video e completamento dei moduli IBM SkillsBuild.

8 Stato di avanzamento e prossimi passi

Tabella 2: Stato delle attività del progetto.

Attività	Stato
Diagnosi del problema di avvio (<code>msvcrt</code> vs <code>pynput</code>)	Completata
Correzione di <code>manual_control.py</code>	Completata
Guida manuale e raccolta del primo giro	Completata
Analisi dei dati e identificazione delle righe da filtrare	Completata
Raccolta di un numero adeguato di giri	Da fare
Filtraggio del dataset	Da fare
Addestramento del modello (K-NN)	Da fare
Script di guida autonoma (<code>auto_drive.py</code>)	Da fare
Pubblicazione su GitHub e video di consegna	Da fare

I prossimi passi immediati sono: l’aumento del numero di giri registrati per arricchire il dataset, l’implementazione del filtraggio dei dati e la realizzazione del modello K-NN, seguita dallo script di guida autonoma. Solo dopo aver ottenuto risultati misurabili in pista (tempi sul giro, assenza di uscite) sarà possibile integrare nel presente documento la sezione dei risultati sperimentali.

9 Conclusioni

Il lavoro svolto ha posto le basi metodologiche e operative del progetto. È stata individuata e risolta la causa del mancato avvio del veicolo, ricondotta a una differenza tecnica nella modalità di lettura della tastiera; è stato corretto e ottimizzato il modulo di guida manuale, con attenzione

alla stabilità, alla velocità e alla qualità dei dati prodotti; è stato raccolto e analizzato il primo giro, identificando le operazioni di pulizia necessarie per la fase successiva. La metodologia di apprendimento — behavioral cloning con modello K-NN, eventualmente esteso a una rete neurale — è stata definita e ne sono state discusse le criticità. Le fasi di addestramento e di guida autonoma, insieme alla pubblicazione finale, costituiscono il lavoro residuo, qui descritto nella sua metodologia e tracciato nel piano di avanzamento.

